

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE CODE: CSA1407**  
**COMPILER DESIGN**

**COURSE NAME:**

---

### COURSE OUTCOME COVERED

**CO3:** Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

*Assessment Tool Weightage: 20%*

---

**QUESTIONS: 5**

**Total Marks:100**

Annexure A

### CASE STUDY

---

#### ***Code of Conduct:***

*I V.Vamsidhar Reddy-192365029 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: V.VAMSI***

| Q. No                 | Understanding of Concepts (25) | Experimental Design (25) | Use of Tools (20) | Analysis of Results (20) | Documentation (10) | Total Score (out of 100) |
|-----------------------|--------------------------------|--------------------------|-------------------|--------------------------|--------------------|--------------------------|
| 1                     | / 4                            | / 4                      | / 4               | / 4                      | / 4                |                          |
| 2                     | / 4                            | / 4                      | / 4               | / 4                      | / 4                |                          |
| 3                     | / 4                            | / 4                      | / 4               | / 4                      | / 4                |                          |
| 4                     | / 4                            | / 4                      | / 4               | / 4                      | / 4                |                          |
| 5                     | / 4                            | / 4                      | / 4               | / 4                      | / 4                |                          |
| OVERALL RUBRICS VALUE |                                |                          |                   |                          |                    |                          |
|                       | / 4                            | / 4                      | / 4               | / 4                      | / 4                |                          |
|                       | / 25                           | / 25                     | / 20              | / 20                     | / 10               | / 100                    |

## Annexure A

### CASE STUDY

#### Set 7

1

A mobile application company is developing a lightweight programming language for smart devices where expressions and assignments must be translated efficiently into intermediate code. The compiler team initially applies syntax-directed definitions with synthesized attributes to build syntax trees for arithmetic operations. As the language expands with nested conditional statements and function calls, inherited attributes become necessary, leading to the use of L-attributed definitions. During testing, semantic errors occur because variables of incompatible types are used together in expressions and assignments. To improve compilation accuracy, the developers introduce a type checker capable of verifying type equivalence and performing safe type conversions.

- a) Explain the role of syntax-directed definitions in syntax tree construction.
- b) How do type checking and type conversion improve semantic correctness?

#### a) Explain the role of Syntax-Directed Definitions in syntax tree construction

**Syntax-Directed Definition (SDD)** is a method used in compiler design to associate **semantic rules and attributes with grammar productions**. It combines syntax analysis with semantic information and helps the compiler understand the meaning of a program construct.

#### Role of SDD in Syntax Tree Construction

1. **Associates semantic rules with grammar**  
Semantic rules are attached to grammar productions to specify how information is calculated.
2. **Uses attributes**  
Attributes store information related to grammar symbols. They may be **synthesized** or **inherited**.
3. **Synthesized attributes**  
Synthesized attributes obtain their values from child nodes and pass the information upward to the parent.
4. **Constructs syntax trees**  
SDD can be used to create nodes for operators and operands, forming a hierarchical syntax tree.
5. **Represents program structure**  
The syntax tree removes unnecessary grammar details and represents the essential structure of an expression.

### Example

Consider the expression:

$a + b * c$

Because multiplication has higher precedence, the syntax tree represents it as:

```

  +
 / \
a  *
   / \
  b  c
```

Here:

- + is the root operator.
- a is the left operand.
- \* is a child of +.
- b and c are operands of \*.

The semantic rules can calculate the values of the child nodes and synthesize the result upward.

For example:

$T \rightarrow T * F$

$\{ T.val = T1.val * F.val \}$

$E \rightarrow E + T$

$\{ E.val = E1.val + T.val \}$

If:

$a = 2$

$b = 3$

$c = 4$

then:

$b * c = 3 * 4 = 12$

$a + 12 = 14$

Therefore:

$E.val = 14$

### Advantages of SDD

- Helps construct syntax trees.
- Associates meaning with grammar rules.
- Supports expression evaluation.
- Helps generate intermediate code.
- Maintains operator precedence.
- Makes semantic analysis easier.
- Reduces unnecessary intermediate computations.

### b) How do Type Checking and Type Conversion improve semantic correctness?

**Type checking** is an important activity of the **semantic analysis phase** of a compiler. It verifies whether the data types used in expressions, assignments, and function calls are compatible.

#### Type Checking

The compiler checks whether an operation is valid for the given operands.

For example:

$\text{int} + \text{int} \rightarrow \text{Valid}$

$\text{int} + \text{float} \rightarrow \text{Valid}$

$\text{int} + \text{char}^* \rightarrow \text{Type Error}$

For an assignment:

```
int x;  
x = 10;  
the types are compatible.  
But:  
int x;  
x = "hello";  
causes a type error because an integer variable cannot directly store a string.
```

### **Type Equivalence**

The compiler may also compare two type expressions to determine whether they are equivalent.

Examples:

```
int and int    → Equivalent  
float and int  → Not Equivalent  
int* and int*  → Equivalent  
int* and float* → Not Equivalent
```

This prevents incompatible types from being used together.

### **Benefits of Type Checking and Conversion**

1. Detects incompatible operations.
2. Prevents invalid assignments.
3. Ensures operands are compatible.
4. Detects errors during compilation.
5. Allows safe implicit conversions.
6. Supports explicit conversions when required.
7. Reduces runtime errors.
8. Improves program reliability.
9. Helps generate correct intermediate code.
10. Ensures the program follows the language's type rules.

### **Example**

Consider:

```
int a = 10;  
float b = 2.5;  
float c = a + b;
```

The compiler can convert a from int to float and then perform:

$10.0 + 2.5 = 12.5$

So the expression is type-compatible.

However:

```
int a;  
char *p;
```

$a + p$ ;

is invalid because an integer and a character pointer cannot be used with + under the language's type rules.

### **Conclusion**

Thus, **Syntax-Directed Definitions** connect grammar with semantic processing and help construct syntax trees and evaluate expressions. **Type checking and type conversion** ensure that operations and assignments use compatible data types. Together, they improve the **semantic correctness, reliability, and safety** of the compiled program.

## 2

A research laboratory is designing a scientific computing language where mathematical expressions must be translated into optimized intermediate representations. Initially, the compiler uses top-down translation schemes for handling simple expressions, but the approach becomes inefficient for deeply nested operations. To overcome this limitation, the developers adopt bottom-up evaluation techniques using syntax-directed definitions. During semantic analysis, the compiler encounters multiple errors due to incompatible type expressions between arrays, integers, and floating-point variables. The team then implements a type system to check type equivalence and apply suitable implicit conversions.

a) Compare top-down and bottom-up translation methods in compiler design.

b) Why is type equivalence important in semantic analysis?

c)

### a) Compare Top-Down and Bottom-Up Translation Methods in Compiler Design

Translation methods determine how a compiler processes grammar rules and performs semantic actions during parsing.

#### 1. Top-Down Translation

In **top-down translation**, processing begins with the **start symbol** and proceeds toward the input symbols or leaf nodes.

The structure is developed from **parent to child**.

Example:

$E \rightarrow T + E$

$T \rightarrow id$

The compiler starts with E and expands it into smaller grammar symbols.

#### Features of Top-Down Translation

- Starts from the start symbol.
- Moves from parent to child.
- Follows a top-to-bottom approach.
- Commonly associated with recursive-descent parsing.
- Can naturally support inherited attributes.
- Suitable for simple grammar structures.

#### 2. Bottom-Up Translation

In **bottom-up translation**, processing starts with the input symbols and gradually combines them until the **start symbol** is obtained.

The structure is developed from **children to parent**.

For example:

$a + b * c$

The compiler first identifies lower-level components such as:

$b * c$

and then combines the result with a +.

#### Features of Bottom-Up Translation

- Starts from input symbols.
- Moves from child to parent.
- Uses reductions to build higher-level structures.
- Commonly associated with LR and shift-reduce parsing.

- Works naturally with synthesized attributes.
- Suitable for complex expressions and grammars.

## Comparison

### Top-Down Translation

Starts from start symbol

Parent → child

Builds structure downward

Commonly uses recursive descent

Can handle inherited information naturally

Uses prediction/expansion

Suitable for simpler grammars

### Bottom-Up Translation

Starts from input symbols

Child → parent

Builds structure upward

Commonly uses LR/shift-reduce parsing

Naturally handles synthesized information

Uses shift and reduce operations

Suitable for more complex grammars

## Example

For:

$a + b * c$

Bottom-up evaluation can first calculate:

$b * c = 12$

and then:

$a + 12 = 14$

This follows the child-to-parent evaluation required by synthesized attributes.

---

## b) Why is Type Equivalence Important in Semantic Analysis?

**Type equivalence** determines whether two type expressions represent the same or compatible type.

It is an important part of **semantic analysis** because the compiler must ensure that variables, expressions, assignments, and function parameters use compatible types.

## Examples

int and int → Equivalent

float and float → Equivalent

int and float → Not Equivalent

int\* and int\* → Equivalent

int\* and float\* → Not Equivalent

## Types of Type Equivalence

### 1. Name Equivalence

Two types are considered equivalent only when they have the **same declared type name**.

Example:

type A = int;

type B = int;

Depending on the language rules, A and B may be treated as different named types.

### 2. Structural Equivalence

Two types are equivalent when their **internal structures are the same**, even if their names are different.

For example, two array types having the same element type and size may be structurally equivalent.

## Importance of Type Equivalence

1. Checks compatibility between data types.
2. Prevents invalid assignments.

3. Helps validate arithmetic operations.
4. Checks function arguments and return types.
5. Helps identify pointer type mismatches.
6. Detects semantic errors during compilation.
7. Improves program reliability.
8. Ensures correct intermediate-code generation.

### Example

Consider:

```
int a;  
float b;
```

```
a = b;
```

The compiler checks the types of a and b. If the language does not allow the assignment directly, a type error is reported or a permitted conversion is applied.

Another example:

```
int* p;  
float* q;
```

Here:

```
int* ≠ float*
```

because their base types are different.

### Type Equivalence and Conversion

When types are compatible but different, the compiler may apply a permitted conversion.

For example:

```
int + float
```

The int value may be converted to float, allowing the arithmetic operation.

### Conclusion

Thus, **top-down translation** processes grammar from the start symbol toward the input, while **bottom-up translation** processes input symbols toward the start symbol. Type equivalence is essential in semantic analysis because it ensures that only **compatible types** are used in expressions, assignments, and function operations, reducing semantic and runtime errors.

A cloud computing company is building a domain-specific language for processing distributed data streams. The compiler constructs syntax trees using syntax-directed translation and evaluates attributes during parsing to generate intermediate code. While S-attributed definitions work effectively for arithmetic expressions, inherited attributes are later required for scope handling and parameter passing in distributed functions. During execution, the compiler reports semantic errors because logical and numeric expressions are mixed incorrectly without type validation. To address these issues, the development team creates a simple type checker with automatic type conversion support.

- a) Differentiate between S-attributed and L-attributed definitions.
- b) How does a type checker enhance compiler reliability?

### a) Differentiate between S-Attributed and L-Attributed Definitions

Attribute grammars are used in compilers to associate semantic information with grammar symbols. The two important types are **S-attributed** and **L-attributed definitions**.

#### S-Attributed Definition

An **S-attributed definition** uses only **synthesized attributes**.

A synthesized attribute gets its value from the attributes of its child nodes and passes the result upward to the parent.

Example:

$E \rightarrow E1 + T$

{  $E.val = E1.val + T.val$  }

Here,  $E.val$  is calculated using  $E1.val$  and  $T.val$ .

#### Features of S-Attributed Definition

- Uses only synthesized attributes.
- Information flows from child to parent.
- Naturally evaluated from bottom to top.
- Suitable for bottom-up parsing.
- Commonly used with LR parsers.
- Useful for arithmetic expression evaluation.

Example:

$3 * 4 = 12$

$2 + 12 = 14$

The values are calculated from the lower-level nodes and passed upward.

#### L-Attributed Definition

An **L-attributed definition** can use both **synthesized and inherited attributes**.

Inherited attributes allow information to be passed from a parent or from symbols on the left side of a production to a child.

Example:

$A \rightarrow X Y$

Information from  $A$  or  $X$  can be passed to  $Y$ .

#### Features of L-Attributed Definition



- Can use synthesized and inherited attributes.
- Information can flow from parent to child.
- Supports left-to-right evaluation.
- Suitable for top-down translation.
- Useful for scope, declarations, parameter passing, and type information.

### Difference Between S-Attributed and L-Attributed

| S-Attributed                     | L-Attributed                                    |
|----------------------------------|-------------------------------------------------|
| Uses only synthesized attributes | Uses synthesized and inherited attributes       |
| Child → parent                   | Parent/left sibling → child and child → parent  |
| Mainly bottom-up                 | Left-to-right evaluation                        |
| Suitable for LR parsing          | Suitable for top-down/left-to-right translation |
| Simpler to implement             | More flexible                                   |
| Useful for expression evaluation | Useful for scope and parameter information      |

---

### b) How Does a Type Checker Enhance Compiler Reliability?

A **type checker** is a component of semantic analysis that verifies whether the types used in a program are compatible with the language rules.  
It checks variables, expressions, assignments, operators, function arguments, and return values.

#### Example

Consider:

```
int a;
int b;
```

```
a + b;
```

This is valid because both operands are integers.

But:

```
int a;
char *p;
```

```
a + p;
```

may produce a **type error** because the operands are incompatible.

### Functions of a Type Checker

#### 1. Checks variable assignments

```
int x;
x = 10;    → Valid
```

#### 2. Checks arithmetic operations

```
int + int    → Valid
int + float  → Usually valid with conversion
int + char*  → Type Error
```

#### 3. Checks function arguments

The type of an argument is compared with the expected parameter type.

#### 4. Checks return values

The returned value must be compatible with the function's declared return type.

#### 5. Checks type equivalence

It determines whether two type expressions are equivalent.

Example:

```
int* and int*    → Equivalent
```

int\* and float\* → Not Equivalent

#### 6. **Detects errors before execution**

Many type errors can be detected during compilation rather than allowing them to cause problems during execution.

#### **Type Conversion**

A type checker can also work with permitted type conversions.

For example:

```
int a = 10;
```

```
float b;
```

```
b = a;
```

The integer value may be converted to a floating-point value when the language permits it.

#### **Benefits**

- Detects incompatible operations.
- Prevents invalid assignments.
- Improves program reliability.
- Reduces runtime errors.
- Ensures correct use of variables.
- Helps generate correct intermediate code.
- Enforces the language's type rules.

#### **Conclusion**

Thus, **S-attributed definitions** use synthesized attributes and are naturally suited to bottom-up evaluation, while **L-attributed definitions** support both synthesized and inherited attributes and allow left-to-right information flow. A **type checker** improves compiler reliability by detecting incompatible types and ensuring that expressions, assignments, and function operations follow the language's type rules.

A university project team is implementing a compiler for a mini educational language that supports loops, conditions, and arithmetic expressions. Although the parser successfully recognizes grammar rules, the semantic analyzer fails to evaluate attributes correctly for nested expressions. To improve translation accuracy, the students implement syntax-directed definitions and use bottom-up evaluation for constructing syntax trees. During testing, the compiler frequently detects invalid operations involving incompatible type expressions such as boolean and floating-point variables. The students therefore add type equivalence checking and safe conversion mechanisms to reduce semantic errors.

- a) How does bottom-up evaluation assist syntax-directed translation?
- b) Why are type systems essential in compiler construction?

#### a) How does Bottom-Up Evaluation assist Syntax-Directed Translation?

**Bottom-up evaluation** processes the input starting from the **leaf nodes** and **moves toward the root**. It evaluates the child nodes before evaluating their parent nodes.

This approach is especially useful for **Syntax-Directed Definitions (SDD)** because synthesized attributes naturally flow from children to parents.

#### Working of Bottom-Up Evaluation

The general process is:

Input → Leaf nodes → Intermediate nodes → Root node

For an arithmetic expression:

$a + b * c$

the compiler first evaluates the lower-level operation:

$b * c$

Then it evaluates:

$a + (b * c)$

If:

$a = 2$

$b = 3$

$c = 4$

then:

$b * c = 3 * 4 = 12$

$a + 12 = 14$

Therefore:

Final result = 14

#### Role in Syntax-Directed Translation

Bottom-up evaluation helps to:

1. Evaluate child nodes before their parents.
2. Calculate synthesized attributes.
3. Construct syntax trees from lower-level components.
4. Generate intermediate code.
5. Handle arithmetic expressions correctly.
6. Work naturally with LR and shift-reduce parsers.
7. Maintain operator precedence.

#### Example of Synthesized Attribute

Consider:

$E \rightarrow E1 + T$

{  $E.val = E1.val + T.val$  }

The values of E1 and T are calculated first. Their values are then used to calculate E.val.

Thus, information flows:

Child nodes  $\rightarrow$  Parent node

This is the basic idea of bottom-up evaluation.

## b) Why are Type Systems Essential in Compiler Construction?

A **type system** defines the rules for assigning and using data types in a programming language.

It ensures that operations are performed only on compatible types.

Type systems are mainly used during **semantic analysis**.

### Example

```
int a = 10;
```

```
int b = 20;
```

```
a + b
```

This is a valid operation because both operands are integers.

However:

```
int a;
```

```
char *p;
```

```
a + p;
```

can produce a **type error** because the operands are incompatible under normal arithmetic rules.

## Importance of Type Systems

### 1. Detects Type Errors

It identifies invalid operations such as:

```
int + char*
```

### 2. Checks Assignments

It verifies that the value being assigned is compatible with the variable's type.

```
int x;
```

```
x = 10;     $\rightarrow$  Valid
```

### 3. Checks Expressions

It ensures operators are applied to suitable operands.

```
int * int     $\rightarrow$  Valid
```

```
float + int    $\rightarrow$  Usually valid with conversion
```

### 4. Checks Type Equivalence

It determines whether two type expressions are equivalent.

```
int* and int*     $\rightarrow$  Equivalent
```

```
int* and float*   $\rightarrow$  Not Equivalent
```

### 5. Supports Type Conversion

When allowed by the language, the compiler can convert one compatible type into another.

Example:

```
int a = 10;
```

```
float b;
```

```
b = a;
```

The integer can be converted to a floating-point value.

## 6. Prevents Runtime Problems

Detecting type errors during compilation prevents many invalid operations from reaching program execution.

## 7. Improves Program Reliability

A strong type system helps ensure that expressions, assignments, functions, and data structures are used correctly.

## Other Uses

Type systems help with:

- Variable declarations
- Arithmetic operations
- Function parameters
- Return values
- Arrays
- Pointers
- Structures
- Assignments
- Type conversions

## Conclusion

Thus, **bottom-up evaluation** helps syntax-directed translation by evaluating child nodes before parent nodes and naturally supports synthesized attributes. A **type system** is essential because it detects incompatible operations, validates assignments and expressions, supports type conversion, and improves the safety and reliability of compiled programs.

5

A software company developing an embedded systems language needs a compiler capable of translating hardware control statements into efficient intermediate code. The compiler initially relies on top-down translation methods, but attribute dependencies in nested statements lead to incorrect syntax tree generation. To improve semantic processing, the developers redesign the compiler using bottom-up evaluation of syntax-directed definitions. During semantic analysis, several runtime risks are identified because incompatible data types are assigned without proper validation. To ensure program safety, the team integrates a type checker that verifies type equivalence and performs explicit and implicit type conversions where necessary.

a) Explain the significance of syntax trees in semantic analysis.

b) How do type conversions help avoid runtime errors?

### a) Explain the Significance of Syntax Trees in Semantic Analysis

A **syntax tree** is a hierarchical representation of a program that shows the relationship between operators, operands, expressions, and statements.

It is constructed after syntax analysis and is widely used during **semantic analysis**.

#### Example

For the expression:

$a + b * c$

the syntax tree is:

```

  +
 / \
a  *
   / \
  b  c
```

Here, + is the root, while \* represents the multiplication operation.

#### Importance of Syntax Trees

##### 1. Represents program structure

It shows the hierarchical relationship between different program elements.

##### 2. Supports semantic analysis

The compiler can examine the meaning and relationships of expressions.

##### 3. Helps in type checking

The compiler can check whether operands of each operator have compatible types.

##### 4. Supports attribute evaluation

Synthesized and inherited attributes can be evaluated using the tree structure.

##### 5. Helps generate intermediate code

The compiler can traverse the tree to produce intermediate representations.

##### 6. Preserves operator precedence

For example, \* is represented below + in:

$a + b * c$

##### 7. Supports optimization

The compiler can identify unnecessary or repeated computations.

##### 8. Simplifies further compilation phases

The tree provides a structured representation instead of the original source text.

## Example of Semantic Analysis

For:

```
int a;
```

```
float b;
```

```
a + b;
```

the syntax tree helps the compiler identify the + operation and check the types of a and b.

## b) How Do Type Conversions Help Avoid Runtime Errors?

**Type conversion** changes a value from one data type to another so that compatible operations or assignments can be performed.

There are two main types:

### 1. Implicit Type Conversion

The compiler automatically converts a value when the language permits it.

Example:

```
int a = 10;
```

```
float b;
```

```
b = a;
```

Here, the integer value can be converted to a floating-point value.

The operation becomes conceptually:

$10 \rightarrow 10.0$

This allows the assignment to be performed correctly.

### 2. Explicit Type Conversion

The programmer explicitly requests the conversion.

Example:

```
float x = 10.5;
```

```
int y;
```

```
y = (int)x;
```

Here, the programmer explicitly converts the floating-point value to an integer.

## How Type Conversion Prevents Errors

1. Makes compatible operands available for operations.
2. Prevents invalid assignments.
3. Handles mixed numeric expressions.
4. Allows different numeric types to work together when permitted.
5. Helps the compiler generate correct intermediate code.
6. Detects unsafe or invalid conversions.
7. Reduces the possibility of type-related runtime errors.

### Example

Consider:

```
int a = 10;
```

```
float b = 2.5;
```

```
a + b;
```

The compiler can convert a to a floating-point value:

```
10.0 + 2.5 = 12.5
```

Therefore, the operation can be performed safely.

### Type Conversion vs Type Error

A compatible conversion:

```
int → float
```

may be allowed.

But an incompatible operation such as:

```
int + char*
```

should produce a **type error** rather than allowing an invalid operation.

### Conclusion

Thus, **syntax trees** provide a structured representation of programs and help the compiler perform type checking, semantic analysis, optimization, and intermediate code generation. **Type conversions** make compatible data types usable together and help prevent invalid assignments and operations, improving the safety and reliability of the p



## RUBRICS FOR CALCULATION

| Criteria                | Weightage | Level 4 – Exemplary                                              | Level 3 – Proficient                           | Level 2 – Developing                    | Level 1 – Beginning                                  |
|-------------------------|-----------|------------------------------------------------------------------|------------------------------------------------|-----------------------------------------|------------------------------------------------------|
| Understanding of Case   | 25        | Thorough understanding; clearly identifies all key issues        | Good understanding with most issues identified | Basic understanding; some issues missed | Poor understanding; problem not identified correctly |
| Application of Concepts | 25        | Effectively applies relevant concepts to analyze the case deeply | Applies appropriate concepts with minor gaps   | Limited application of concepts         | Fails to apply relevant concepts                     |
| Analysis                | 20        | In-depth analysis with strong reasoning and insights             | Good analysis with logical reasoning           | Basic analysis with limited depth       | Weak or no analysis; lacks reasoning                 |
| Solution Design         | 20        | Innovative, feasible solutions with strong justification         | Logical solutions with adequate justification  | Basic solutions with weak justification | Incorrect or no solution provided                    |
| Clarity                 | 10        | Clear, well-structured, and easy to understand                   | Organized and understandable presentation      | Limited clarity and structure           | Poor clarity; difficult to understand                |

